

ROADMAP — SISTEMA DE CINE

TP PROGRAMACIÓN IV · DOCUMENTO VIVO

Plataforma de cine completa: catálogo público, selección de butacas, checkout con Candy Bar/combos/cupones, fidelización, tickets QR, panel de administrador y panel de empleado para validación presencial. Se avanza tarea por tarea, con diseño visual único (estética oscura, motivos de cine) y soporte para compra anónima y registrada desde el catálogo público.

DECISIONES CONFIRMADAS

DECISIÓN	ELEGIDO
Estilo visual	SCSS propio con design tokens (paleta oscura tipo sala de cine). Sin Angular Material ni Tailwind.
Pagos	Checkout con pasarela simulada (mock), persistida igual en la tabla pagos como aprobada.
Despliegue	Vercel
Componentes	Standalone por defecto. compra y administracion usan NgModule clásico; el resto sigue standalone.
Manejo de estado	Signals de Angular + servicios inyectables. Sin NgRx.
Acceso a datos	SupabaseService central + servicios de dominio por feature. Nunca Supabase directo desde un componente.
Concurrencia de butacas	Validación de disponibilidad vía transacción/RPC en Postgres, no solo en el frontend.
Commits / PRs	Sin líneas de atribución al agente. Toda PR incluye Objetivo inicial / Qué se terminó haciendo / Resumen de cambios.
Idioma del código	Todo en español (componentes, servicios, rutas, variables) salvo API de Angular/TS/RxJS y vocabulario técnico de arquitectura.
Tests unitarios	No se generan salvo pedido explícito.
Archivos de componente	Siempre 3 separados (.ts/.html/.css), nunca template/estilos inline.

ESTRUCTURA DE CARPETAS

```
src/app/  
  core/          # SupabaseService, guards, interceptors, modelos TS compartidos  
  shared/        # UI primitives propias (botón, card, badge, spinner, modal...)  
  layout/        # header, footer, shell con outlet  
  features/  
    catalogo/    # M02 – listado, detalle, búsqueda/filtro, reseñas  
    salas-butacas/ # M03/M04 – mapa de butacas, selección, disponibilidad  
    compra/      # M05/M06/M07 – carrito, candy/combos, cupones, pago mock (NgModule)  
    fidelizacion/ # M08 – puntos, crédito, canjes  
    entradas/    # M09 – ticket PDF + QR (vista cliente)  
    cancelaciones/ # M10  
    proxíamente/ # M11 – estrenos, alertas
```

```
perfil/      # M01 - registro, login, datos del cliente, Mis películas
empleado/    # validación QR/código manual
administracion/ # M12/M13 + ABM de todo lo anterior (NgModule)
app.routes.ts # composición de rutas lazy por feature
```

ROADMAP POR FASES

Fase 0 — Fundaciones del proyecto

✓ Hecho

- 0.1 Limpieza de repo y documentación base (README + CHANGELOG) ✓
- 0.2 Estructura de carpetas (features/* + rutas lazy placeholder) ✓
- 0.3 Supabase: cliente, environment.ts, SupabaseService ✓
- 0.4 Design system base: variables SCSS + componentes Button/Card ✓
- 0.5 Shell de layout: header, footer, fondo temático ✓
- 0.6 Esquema SQL inicial en Supabase (tablas base + RLS) ✓
- 0.7 Seed de datos de demo ✓

Fase 1 — Catálogo público (M02)

🔄 En progreso

Listado de películas (poster, nombre, duración, clasificación), detalle (sinopsis, géneros, reseñas con estrellas y promedio, funciones disponibles), buscador + filtro múltiple por género, destacado "3 más vendidas" en home.

- 1.1 Capa de datos: modelos + PeliculasService (listado/detalle/destacadas) + RPC agregada de "más vendidas" ✓
- 1.2 Página de listado (home): grilla de películas + destacado "3 más vendidas" ✓
- 1.3 Buscador + filtro múltiple por género ✓
- 1.4 Página de detalle de película (sinopsis, géneros, funciones disponibles, reseñas + promedio) ☐

Fase 2 — Salas, butacas y funciones (M03/M04)

☐ Pendiente

Mapa visual de butacas (normal/accesible/VIP con diferenciación de color clara), disponibilidad en tiempo real, selección de función desde el detalle.

Fase 3 — Autenticación y perfil (M01)

☐ Pendiente

Registro (mail, nombre, apellido, fecha nacimiento, tipo de sangre, color de ojos, vacaciones anuales), login/logout, perfil (datos, puntos, crédito, historiales) — sin romper la navegación de invitado.

Fase 4 — Compra: entradas + Candy/combos + cupones (M05/M06/M07)

☐ Pendiente

Carrito (entradas + productos + combos), cupón, compra anónima vs registrada, validación de edad (RN04), pantalla de pago simulada, persistencia de venta.

Fase 5 — Entradas y QR (M09)

☐ Pendiente

PDF de entrada + QR, pantalla "Mis entradas".

Fase 6 — Panel de empleado: validación (M09 operación)

☐ Pendiente

Escaneo/ingreso manual de código, validación de entrada y retiro de Candy, invalidación de QR usado.

Fase 7 — Panel de administración: ABM base (M03/M04/M06/M07 admin)

☐ Pendiente

CRUD de películas, salas/butacas, funciones (asignación automática de sala + RN01/RN02/RN03), productos/categorías, combos, cupones y preventa.

Fase 8 — Fidelización y crédito (M08)

☐ Pendiente

Acreditación de puntos, configuración y canje de recompensas, saldo e historial, no transferencia.

Fase 9 — Cancelaciones (M10)

☐ Pendiente

Cancelación hasta 2 horas antes, liberación de butacas, generación de crédito.

Fase 10 — Próximamente y notificaciones (M11)

☐ Pendiente

Sección "Próximamente", alertas de estreno, preventa 7 días antes, notificación in-app al habilitarse la venta.

Fase 11 — Reportes y estadísticas (M12)

☐ Pendiente

Facturación diaria, entradas vendidas, exportación PDF/Excel, gráficos de películas más vistas y producto más vendido.

Fase 12 — Auditoría (M13)

☐ Pendiente

Registro y consulta de logs de acciones administrativas y de validación.

Fase 13 — PWA y pulido final

☐ Pendiente

Manifest + service worker, instalabilidad, revisión de RLS, accesibilidad, responsive final, despliegue a Vercel, README de arquitectura (RNF-008).

PUNTOS ABIERTOS

Se resuelven al llegar a la fase correspondiente.

Edad en compra anónima (Fase 4) — resuelto: el checkout anónimo pide fecha de nacimiento y se valida igual que con un perfil registrado.

Canal de notificaciones de estreno (Fase 10) — arrancar in-app, evaluar email después.

Borrado físico vs soft-delete (Fase 7) — se propone soft-delete (activo/activa) salvo indicación contraria.

Layout gráfico de butacas accesibles (Fase 2) — se define con un mockup simple antes de codear.

Cada PR agrega una entrada nueva en CHANGELOG.md · El README se actualiza cuando una fase introduce una decisión de arquitectura nueva
· Cada PR describe Objetivo inicial / Qué se terminó haciendo / Resumen de cambios, sin líneas de atribución al agente.